

BFS, DFS & Graph Algorithms — Complete Placement + CP Guide

Target: SDE placements, online assessments, competitive programming

Covers: BFS · DFS · Connected Components · Cycle Detection (Undirected + Directed) · Bipartite Graph · Flood Fill · Matrix BFS/DFS · Multi-Source BFS · 0-1 BFS · DFS Timestamps · Bridges · Articulation Points

BFS (Breadth-First Search)

1. Overview

BFS is a graph traversal algorithm that explores vertices **level by level**. It starts at a source node and visits all its neighbours first, then all neighbours of neighbours, and so on. It uses a **queue** to maintain the order of exploration.

2. Intuition

Think of it like a **tsunami wave** spreading outward from the epicentre. The wave reaches all points at distance 1 first, then all points at distance 2, and so on.

Step-by-step reasoning:

1. Start at the source node, mark it visited.
2. Push it into a queue.
3. While the queue is not empty, pop the front node.
4. Visit all its unvisited neighbours, mark them visited, push them into the queue.
5. Repeat.

Why it works: The queue ensures FIFO ordering, so nodes closer to the source (with smaller distance) are processed before farther nodes. This guarantees that BFS finds the **shortest path** in an unweighted graph.

3. When to Use It

- Shortest path in an **unweighted** graph
- Level-order traversal of a tree
- Finding connected components
- Finding if a graph is bipartite
- Solving puzzles with uniform move cost (maze, word ladder)
- Minimum number of steps/operations to reach a state
- **Trigger phrases:** "shortest path", "minimum steps", "minimum moves", "level order", "nearest", "distance in grid", "word ladder"

4. When Not to Use It

- Graph is **weighted** (use Dijkstra or 0-1 BFS)

- Graph is **very deep** and branching factor is large (BFS memory blows up)
- Only need to know **if** a path exists, not the shortest (DFS is simpler)
- Graph is infinite or extremely large (DFS with iterative deepening may be better)
- **Overkill:** For a simple reachability check in a small graph, DFS is simpler

5. Core Concepts

5.1 Queue

- The engine of BFS. Nodes are pushed when first discovered and popped when processed.
- **Why it matters:** FIFO property ensures level-order traversal.

5.2 Visited Array

- A boolean array/set that marks nodes once visited.
- **Why it matters:** Prevents infinite loops and revisiting nodes.

5.3 Distance Array

- Stores the shortest distance from source to each node.
- **Why it matters:** BFS naturally computes shortest distances in unweighted graphs.

5.4 Parent Array

- Stores the predecessor of each node in the BFS tree.
- **Why it matters:** Allows reconstructing the shortest path.

5.5 Level

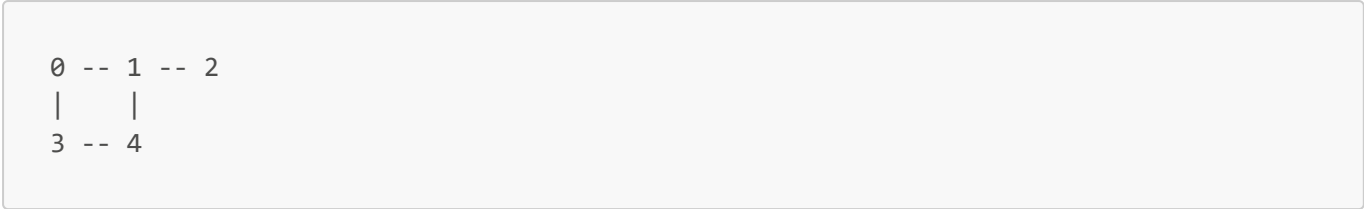
- The distance from the source. All nodes at the same level are at the same distance.
- **Why it matters:** BFS processes nodes in non-decreasing order of distance.

6. Step-by-Step Algorithm

1. Create a queue `q`.
2. Create a visited array `vis` of size `n`, initialized to `false`.
3. Create a distance array `dist` of size `n`, initialized to `INF` (or `-1`).
4. Mark source `s` as visited, set `dist[s] = 0`, push `s` into `q`.
5. While `q` is not empty:
 - Pop front node `u`.
 - For each neighbour `v` of `u`:
 - If `v` is not visited:
 - Mark `v` as visited.
 - Set `dist[v] = dist[u] + 1`.
 - Push `v` into `q`.
6. After the loop, `dist` contains shortest distances from `s` to all reachable nodes.

7. Dry Run

Graph:



Edges: (0-1), (1-2), (0-3), (1-4), (3-4)

Source: 0

Step	Queue (front → back)	Visited	dist[0]	dist[1]	dist[2]	dist[3]	dist[4]
Init	[0]	{0}	0	∞	∞	∞	∞
1	Pop 0 → push 1,3	{0,1,3}	0	1	∞	1	∞
2	Pop 1 → push 2,4	{0,1,3,2,4}	0	1	2	1	2
3	Pop 3 → no new	{0,1,3,2,4}	0	1	2	1	2
4	Pop 2 → no new	{0,1,3,2,4}	0	1	2	1	2
5	Pop 4 → no new	{0,1,3,2,4}	0	1	2	1	2

Result: Shortest distances: `dist = [0, 1, 2, 1, 2]`

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// BFS: returns vector of shortest distances from source
vector<int> bfs(int n, vector<vector<int>> &adj, int source) {
    vector<int> dist(n, -1); // -1 means unreachable
    queue<int> q;

    dist[source] = 0;
    q.push(source);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) { // not visited
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }

    return dist;
}
```

```
// BFS with path reconstruction
vector<int> bfs_path(int n, vector<vector<int>> &adj, int source, int target) {
    vector<int> dist(n, -1), parent(n, -1);
    queue<int> q;

    dist[source] = 0;
    q.push(source);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        if (u == target) break;

        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
    }

    // Reconstruct path
    vector<int> path;
    if (dist[target] == -1) return path; // no path

    for (int v = target; v != -1; v = parent[v])
        path.push_back(v);
    reverse(path.begin(), path.end());
    return path;
}

int main() {
    int n = 5;
    vector<vector<int>> adj(n);
    adj[0] = {1, 3};
    adj[1] = {0, 2, 4};
    adj[2] = {1};
    adj[3] = {0, 4};
    adj[4] = {1, 3};

    vector<int> dist = bfs(n, adj, 0);
    for (int i = 0; i < n; i++)
        cout << "dist[" << i << "] = " << dist[i] << "\n";

    vector<int> path = bfs_path(n, adj, 0, 2);
    cout << "Path 0→2: ";
    for (int v : path) cout << v << " ";
    cout << "\n";
    return 0;
}
```

9. Python Implementation

```
from collections import deque

def bfs(n, adj, source):
    """Return list of shortest distances from source."""
    dist = [-1] * n
    q = deque()

    dist[source] = 0
    q.append(source)

    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

def bfs_path(n, adj, source, target):
    """Return shortest path from source to target."""
    dist = [-1] * n
    parent = [-1] * n
    q = deque()

    dist[source] = 0
    q.append(source)

    while q:
        u = q.popleft()
        if u == target:
            break
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                parent[v] = u
                q.append(v)

    # Reconstruct path
    if dist[target] == -1:
        return []
    path = []
    v = target
    while v != -1:
        path.append(v)
        v = parent[v]
    return path[::-1]

if __name__ == "__main__":
```

```

n = 5
adj = [[] for _ in range(n)]
adj[0] = [1, 3]
adj[1] = [0, 2, 4]
adj[2] = [1]
adj[3] = [0, 4]
adj[4] = [1, 3]

dist = bfs(n, adj, 0)
for i, d in enumerate(dist):
    print(f"dist[{i}] = {d}")

path = bfs_path(n, adj, 0, 2)
print("Path 0→2:", path)

```

10. Code Explanation

- **Queue:** `queue<int>` in C++, `deque` in Python — used for FIFO processing.
- **Dist array:** Initialized to `-1` (unvisited). When a node is first discovered, its distance is set to `dist[parent] + 1`.
- **Parent array:** Stores the previous node. Used for path reconstruction. Start from `target`, follow `parent` pointers back to `source`, then reverse.
- **BFS loop:** Process each node, check all its neighbours. If a neighbour is unvisited, update its distance, set its parent, and push it into the queue.
- **Path reconstruction:** After BFS, walk backwards from target using `parent` array, then reverse the path.

11. Complexity Analysis

Operation	Time	Space
BFS traversal	$O(V + E)$	$O(V)$ for queue + visited/dist
Path reconstruction	$O(V)$	$O(V)$ for parent
Overall	$O(V + E)$	$O(V)$

Best case: $O(V)$ — star graph, source at centre

Worst case: $O(V + E)$ — dense graph, full traversal

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Shortest path in unweighted graph	"minimum edges", "shortest path"	Standard BFS with dist array	LeetCode 1091, GeekforGeeks Shortest Path
Level-order traversal	"level order", "print by level"	Process queue size in inner loop	LeetCode 102, 107

Pattern	How to Identify	Approach	Example Problems
Word ladder	"transform word", "one letter change"	BFS on graph of words differing by 1 char	LeetCode 127
Rotting oranges	"rotten", "spread", "time to infect all"	Multi-source BFS	LeetCode 994
Nearest distance (0/1 matrix)	"nearest 0", "distance to nearest 1"	Multi-source BFS from all targets	LeetCode 542
Minimum steps in maze	"minimum moves", "grid", "shortest path in grid"	BFS on grid with 4-directional moves	LeetCode 1926, GFG Rat in a Maze

13. Common Mistakes

- **Using recursion** — BFS is iterative by nature; recursion is wrong.
- **Using a stack instead of queue** — that gives DFS, not BFS.
- **Not marking visited when pushing** — marking only when popping causes duplicate pushes and infinite loops.
- **Forgetting to handle disconnected components** — BFS from one source only covers one component.
- **Using `int` for distance where graph has $> 2^{31}$ nodes** — rare but watch for overflow in distances.
- **Not resetting visited between multiple BFS calls** — always reinitialize.
- **Modifying the graph during BFS** — can corrupt traversal.

14. Edge Cases

- Empty graph ($n = 0$)
- Single node, no edges
- Disconnected graph (source in one component, target in another)
- Complete graph (all nodes connected to all others)
- Source == target (distance 0)
- Graph with self-loops (should be handled by visited check)
- Multiple edges between same nodes (adjacency list handles this fine)
- Very large graph ($O(V+E)$ still runs, but watch recursion limit in Python)

15. Variations

15.1 Bidirectional BFS

- **What changes:** BFS runs simultaneously from source and target, meeting in the middle.
- **When used:** Large graphs where search space is huge. Reduces complexity from $O(b^d)$ to $O(b^{d/2})$.
- **Placement importance:** Medium — good to know.

15.2 0-1 BFS

- **What changes:** Uses deque instead of queue. Push to front for 0-weight edges, back for 1-weight.
- **When used:** Graph with edge weights 0 or 1.
- **Placement importance:** High — covered later in this guide.

15.3 Multi-source BFS

- **What changes:** Multiple starting nodes in the initial queue.
- **When used:** "Nearest" problems with multiple sources.
- **Placement importance:** High — very common.

15.4 Level-order BFS

- **What changes:** Process nodes level by level using an inner loop over `q.size()`.
- **When used:** Tree level-order traversal, problems requiring grouping by distance.
- **Placement importance:** High.

16. Related Algorithms/Data Structures

Algorithm	Relation	When to Choose
DFS	Alternative traversal	BFS when shortest path, DFS when exploring all paths
Dijkstra	BFS for weighted graphs	BFS when weights are same/unweighted, Dijkstra when weighted
0-1 BFS	Specialised BFS	When edge weights are only 0 or 1
Bellman-Ford	Negative weight shortest path	BFS cannot handle negative weights
A*	Heuristic-based BFS	When you have a good heuristic for the target

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
1091. Shortest Path in Binary Matrix	LeetCode	BFS on grid with 8-directional moves	Easy
1926. Nearest Exit from Entrance in Maze	LeetCode	BFS in grid, stop at boundary	Easy

Medium

Problem	Platform	Idea	Difficulty
127. Word Ladder	LeetCode	BFS on implicit graph of words	Medium
994. Rotting Oranges	LeetCode	Multi-source BFS with time tracking	Medium
542. 01 Matrix	LeetCode	Multi-source BFS from all 0s	Medium

Hard

Problem	Platform	Idea	Difficulty
1263. Minimum Moves to Move a Box to Their Target Location	LeetCode	BFS + state tracking (box + player)	Hard
Labyrinth	CSES	BFS with path reconstruction in grid	Hard
Knight Moves	SPOJ	BFS on chessboard	Hard

18. Interview Explanation

"BFS is a graph traversal algorithm that explores nodes level by level using a queue. I use it when I need the shortest path in an unweighted graph, like minimum steps in a maze or shortest transformation sequence. The key idea is that the first time BFS discovers a node, it has found the shortest path to it. I maintain a queue, a visited array, and a distance array. Each node is pushed into the queue once — when first discovered. The time complexity is $O(V+E)$ and space is $O(V)$. For path reconstruction, I store a parent array."

19. Revision Notes

- BFS → queue, level-order traversal, shortest path in unweighted graph
- Mark visited **when pushing**, not when popping
- Time: $O(V+E)$, Space: $O(V)$
- Use `dist = -1` for unvisited, `dist[neighbour] = dist[current] + 1`
- Path reconstruction: parent array, walk backwards, then reverse
- Multi-source BFS: push all sources into queue initially
- Bidirectional BFS: search from both ends → $O(b^{(d/2)})$

20. Final Cheat Sheet

Aspect	Detail
When to use	Shortest path in unweighted graph, level-order, nearest distance
Data structure	Queue
Time	$O(V + E)$
Space	$O(V)$
Key code pattern	<pre>q.push(src); while(!q.empty()){u=q.front(); q.pop(); for(v: adj[u]) if(!vis[v]){vis[v]=1; dist[v]=dist[u]+1; q.push(v);}}</pre>
Edge cases	Disconnected graph, single node, source == target, self-loops

DFS (Depth-First Search)

1. Overview

DFS is a graph traversal algorithm that explores as far as possible along each branch before **backtracking**. It uses a **stack** (either explicitly or via recursion) to keep track of the path.

2. Intuition

Think of it like exploring a **maze**: you walk down one path until you hit a dead end, then backtrack to the last junction and try a different path.

Step-by-step reasoning:

1. Start at the source node, mark it visited.
2. For each unvisited neighbour, recursively call DFS on it.
3. When all neighbours of a node are visited, backtrack to the previous node.
4. Repeat until all reachable nodes are visited.

Why it works: The stack (call stack or explicit) ensures that the algorithm goes deep first. The visited array prevents revisiting nodes.

3. When to Use It

- Exploring all paths / all possibilities (backtracking)
- Detecting cycles in graphs
- Topological sorting (DAG)
- Finding connected components
- Path finding (when you need **any** path, not shortest)
- Solving puzzles with constraints (N-Queens, Sudoku)
- Tree traversals (preorder, inorder, postorder)
- Finding strongly connected components (Kosaraju / Tarjan)
- **Trigger phrases:** "all paths", "cycle detection", "topological order", "backtracking", "connected components", "reachability"

4. When Not to Use It

- Shortest path in unweighted graph (use BFS — DFS can find a longer path first)
- Graph is very deep (recursion stack overflow in Python)
- Need level-order processing (BFS is natural)
- Graph is very wide and shallow (BFS is simpler)
- **Overkill:** For simple reachability in a small graph, BFS is equally fine but DFS recursion is simpler to write

5. Core Concepts

5.1 Stack (Recursion / Explicit)

- Recursion uses the call stack. Explicit stack gives more control.
- **Why it matters:** Determines traversal order. Recursive DFS is simpler; iterative DFS avoids stack overflow.

5.2 Visited Array

- Marks nodes when first discovered.
- **Why it matters:** Prevents infinite loops.

5.3 Backtracking

- When a node has no more unvisited neighbours, the algorithm returns to the previous node.
- **Why it matters:** Allows exploring all paths.

5.4 DFS Tree / Forest

- The set of edges traversed during DFS forms a tree (or forest for disconnected graphs).
- **Why it matters:** Used for finding bridges, articulation points, SCCs.

5.5 Entry / Exit Times (Timestamps)

- `tin[u]` = time when node `u` is first discovered.
- `tout[u]` = time when node `u` is fully processed.
- **Why it matters:** Critical for advanced graph algorithms (bridges, articulation points, SCC).

6. Step-by-Step Algorithm

Recursive DFS:

1. Mark `u` as visited.
2. For each neighbour `v` of `u`:
 - If `v` is not visited, recursively call `dfs(v)`.
3. Return (backtrack).

Iterative DFS (using explicit stack):

1. Create a stack `st`.
2. Push `(source, 0)` onto stack — pair of node and next neighbour index.
3. Mark source as visited.
4. While stack is not empty:
 - Peek at top `(u, i)`.
 - If `i < adj[u].size()`:
 - Let `v = adj[u][i]`, increment `i` at top.
 - If `v` not visited, mark visited, push `(v, 0)`.
 - Else:
 - Pop from stack (backtrack).

7. Dry Run

Graph: 0 — 1 — 2 — 3

Recursive DFS from 0:

Call	Node	Action	Visited	Stack depth
<code>dfs(0)</code>	0	Visit 0, go to neighbour 1	{0}	1

Call	Node	Action	Visited	Stack depth
dfs(1)	1	Visit 1, go to neighbour 2	{0,1}	2
dfs(2)	2	Visit 2, go to neighbour 3	{0,1,2}	3
dfs(3)	3	Visit 3, no unvisited neighbours, return	{0,1,2,3}	4
Back to 2	2	No more neighbours, return	—	3
Back to 1	1	No more neighbours, return	—	2
Back to 0	0	No more neighbours, return	—	1

Traversal order: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Recursive DFS
void dfs_recursive(int u, vector<vector<int>> &adj, vector<bool> &vis) {
    vis[u] = true;
    cout << u << " ";

    for (int v : adj[u]) {
        if (!vis[v]) {
            dfs_recursive(v, adj, vis);
        }
    }
}

// Iterative DFS (explicit stack)
void dfs_iterative(int source, vector<vector<int>> &adj) {
    int n = adj.size();
    vector<bool> vis(n, false);
    stack<int> st;

    vis[source] = true;
    st.push(source);

    while (!st.empty()) {
        int u = st.top();
        st.pop();
        cout << u << " ";

        // Push in reverse order to maintain same order as recursive
        for (int i = adj[u].size() - 1; i >= 0; i--) {
            int v = adj[u][i];
            if (!vis[v]) {
                vis[v] = true;
                st.push(v);
            }
        }
    }
}
```

```

    }
}
}

// DFS that handles disconnected graphs
void dfs_full(int n, vector<vector<int>> &adj) {
    vector<bool> vis(n, false);
    for (int i = 0; i < n; i++) {
        if (!vis[i]) {
            dfs_recursive(i, adj, vis);
        }
    }
}

int main() {
    int n = 4;
    vector<vector<int>> adj(n);
    adj[0] = {1};
    adj[1] = {0, 2};
    adj[2] = {1, 3};
    adj[3] = {2};

    cout << "Recursive DFS: ";
    vector<bool> vis(n, false);
    dfs_recursive(0, adj, vis);
    cout << "\n";

    cout << "Iterative DFS: ";
    dfs_iterative(0, adj);
    cout << "\n";

    return 0;
}

```

9. Python Implementation

```

def dfs_recursive(u, adj, vis):
    vis[u] = True
    print(u, end=" ")
    for v in adj[u]:
        if not vis[v]:
            dfs_recursive(v, adj, vis)

def dfs_iterative(source, adj):
    n = len(adj)
    vis = [False] * n
    stack = [source]
    vis[source] = True

```

```

while stack:
    u = stack.pop()
    print(u, end=" ")
    # Push in reverse order for same order as recursive
    for v in reversed(adj[u]):
        if not vis[v]:
            vis[v] = True
            stack.append(v)

def dfs_full(n, adj):
    vis = [False] * n
    for i in range(n):
        if not vis[i]:
            dfs_recursive(i, adj, vis)

if __name__ == "__main__":
    n = 4
    adj = [[] for _ in range(n)]
    adj[0] = [1]
    adj[1] = [0, 2]
    adj[2] = [1, 3]
    adj[3] = [2]

    print("Recursive DFS: ", end="")
    dfs_recursive(0, adj, [False] * n)
    print()

    print("Iterative DFS: ", end="")
    dfs_iterative(0, adj)
    print()

```

10. Code Explanation

- **Recursive DFS:** Simple, mirrors the mathematical definition. Risk of stack overflow for deep graphs.
- **Iterative DFS:** Uses explicit stack. The order of neighbours is reversed when pushing to match recursive order.
- **Visited array:** Prevents cycles. Marked **before** pushing onto stack (in iterative) to avoid duplicates.
- **Full DFS:** Loop over all nodes to handle disconnected components.
- **Backtracking** is implicit in recursion (return from function) and explicit in iterative (pop from stack).

11. Complexity Analysis

Operation	Time	Space
DFS traversal	$O(V + E)$	$O(V)$ — visited array + recursion stack (worst-case $O(V)$)
Full DFS (disconnected)	$O(V + E)$	$O(V)$
Iterative DFS	$O(V + E)$	$O(V)$ — explicit stack

Recursion stack worst-case: $O(V)$ for a chain graph (linear).

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
All paths	"find all paths", "all possible routes"	DFS + backtracking, store path when reaching target	LeetCode 797, CSES
Cycle detection	"detect cycle", "is there a cycle"	DFS with recursion stack / visited states	LeetCode 207, 210
Topological sort	"dependency order", "prerequisite order"	DFS with post-order (push after processing)	LeetCode 210
Connected components	"number of islands", "connected components"	DFS full traversal, count calls	LeetCode 200
Backtracking puzzles	"N-Queens", "Sudoku", "combination sum"	DFS with state modification and undo	LeetCode 51, 46, 39
Tree traversals	"preorder/inorder/postorder"	DFS with different order of visiting	LeetCode 144, 94, 145

13. Common Mistakes

- **Not marking visited** — causes infinite loops.
- **Marking visited only after processing children** — causes repeated work.
- **Stack overflow** in recursion for deep graphs (use iterative DFS).
- **Not resetting state** in backtracking problems — always undo changes.
- **Confusing BFS and DFS** — using a queue for DFS or a stack for BFS.
- **Not handling disconnected graphs** — DFS from one source misses components.
- **Modifying the graph** during traversal.

14. Edge Cases

- Empty graph ($n = 0$)
- Single node
- Disconnected graph (multiple components)
- Complete graph
- Graph with self-loops
- Graph with parallel edges
- Deep chain (recursion depth risk)
- Tree (acyclic connected graph)

15. Variations

15.1 Iterative DFS (Explicit Stack)

- **What changes:** Uses `stack` instead of recursion.

- **When used:** Avoid stack overflow in deep graphs.
- **Placement importance:** High — some interviewers prefer iterative.

15.2 DFS with Timestamps

- **What changes:** Record **tin** and **tout** times.
- **When used:** Bridges, articulation points, SCC (Tarjan).
- **Placement importance:** High — covered in detail later.

15.3 Backtracking (DFS with State)

- **What changes:** Modify state before recursive call, undo after.
- **When used:** Combinatorial problems, puzzles.
- **Placement importance:** Very high.

15.4 Iterative Deepening DFS (IDDFS)

- **What changes:** DFS with depth limit, repeated with increasing limit.
- **When used:** Large search space where BFS memory is too high.
- **Placement importance:** Low.

16. Related Algorithms/Data Structures

Algorithm	Relation	When to Choose
BFS	Alternative traversal	BFS for shortest path, DFS for exploring all paths
Backtracking	DFS + state undo	When constraints/pruning are needed
Topological Sort	DFS-based	For DAG ordering
Tarjan's / Kosaraju's	DFS-based	For SCC
DSU	Alternative for connectivity	DSU for dynamic connectivity, DFS for static

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
200. Number of Islands	LeetCode	DFS on grid, count components	Easy
1971. Find if Path Exists in Graph	LeetCode	Simple DFS reachability	Easy

Medium

Problem	Platform	Idea	Difficulty
207. Course Schedule	LeetCode	Cycle detection in directed graph (DFS)	Medium
797. All Paths From Source to Target	LeetCode	DFS + backtracking, store all paths	Medium

Problem	Platform	Idea	Difficulty
130. Surrounded Regions	LeetCode	DFS from boundary	Medium

Hard

Problem	Platform	Idea	Difficulty
51. N-Queens	LeetCode	DFS + backtracking with constraints	Hard
1192. Critical Connections in a Network	LeetCode	DFS timestamps, bridges	Hard
Building Teams	CSES	DFS bipartite check	Hard

18. Interview Explanation

"DFS is a graph traversal that goes as deep as possible before backtracking. I use it for exploring all paths, detecting cycles, topological sorting, and connectivity problems. The core idea is recursion (or an explicit stack) with a visited array. For cycle detection, I track nodes in the current recursion stack. DFS time complexity is $O(V+E)$ and space is $O(V)$ for the visited array plus recursion stack depth. For deep graphs, I prefer iterative DFS to avoid stack overflow."

19. Revision Notes

- DFS → stack (recursion or explicit), explore one path fully before backtracking
- Recursive DFS is simpler; iterative avoids stack overflow
- Mark visited **before** recursive call / pushing to stack
- Full DFS: loop over all nodes for disconnected graphs
- Time: $O(V+E)$, Space: $O(V)$
- For backtracking: modify state, recurse, **undo** modification
- Cycle detection: track **vis** states — 0=unvisited, 1=in current path, 2=done

20. Final Cheat Sheet

Aspect	Detail
When to use	All paths, cycle detection, topological sort, connectivity, backtracking
Data structure	Stack (recursion / explicit)
Time	$O(V + E)$
Space	$O(V)$
Key code pattern	<pre>void dfs(u){vis[u]=1; for(v:adj[u]) if(!vis[v]) dfs(v);}</pre>
Edge cases	Deep chain (stack overflow), disconnected graph, self-loops

Connected Components

1. Overview

A **connected component** is a maximal set of vertices in an undirected graph such that every pair of vertices in the set is connected by a path. Finding connected components means identifying all such groups in a graph.

2. Intuition

Think of a graph as a collection of **islands** (components). Within each island, you can travel from any node to any other node. Between islands, there is no path. Finding connected components is like counting the number of islands.

Step-by-step reasoning:

1. Start from any unvisited node.
2. Run BFS/DFS to visit all nodes reachable from it.
3. All nodes visited in this traversal belong to one component.
4. Repeat from the next unvisited node.
5. Each time you start a new traversal, you've found a new component.

3. When to Use It

- Counting or identifying disconnected groups in a graph
- Finding islands in a grid
- Checking if a graph is connected (single component)
- Preprocessing for graph algorithms that need to run per component
- **Trigger phrases:** "number of islands", "connected components", "disconnected graphs", "provinces"

4. When Not to Use It

- **Dynamic connectivity** (edges added/removed over time) — use DSU
- Need to know if two specific nodes are connected (DSU is simpler for queries)
- Graph is very large and you only need component size (DSU is faster)
- Directed graph — use **strongly connected components** (Kosaraju/Tarjan) instead

5. Core Concepts

5.1 Component

- A maximal set of mutually reachable vertices.
- **Why it matters:** Problems often ask to count components or process each component independently.

5.2 Component ID

- An integer label assigned to each component during traversal.
- **Why it matters:** Allows grouping nodes by component.

5.3 Component Size

- Number of vertices in the component.
- **Why it matters:** Some problems ask for the largest/smallest component.

6. Step-by-Step Algorithm

- 1. Initialize `vis` array of size `n` with `false`.
- 2. Initialize `comp_id` array of size `n` with `-1`.
- 3. Initialize `component_count = 0`.
- 4. For each node `u` from 0 to `n-1`:
 - If `vis[u]` is false:
 - Increment `component_count`.
 - Run BFS/DFS from `u`, marking all visited nodes with `comp_id = component_count - 1`.
- 5. After the loop, `component_count` is the number of components.

7. Dry Run

Graph: 0 — 1 2 3 — 4

Step	Node	Action	vis	comp_id	component_count
1	0	Start traversal, comp=0	{0,1}	[0,0,-1,-1,-1]	1
2	2	Skip visited, start traversal comp=1	{0,1,2}	[0,0,1,-1,-1]	2
3	3	Start traversal comp=2	{0,1,2,3,4}	[0,0,1,2,2]	3

Components: {0,1}, {2}, {3,4}

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Returns component id for each node and total count
pair<vector<int>, int> connected_components(int n, vector<vector<int>> &adj) {
    vector<int> comp_id(n, -1);
    int comp_count = 0;

    function<void(int, int)> dfs = [&](int u, int id) {
        comp_id[u] = id;
        for (int v : adj[u]) {
            if (comp_id[v] == -1) {
                dfs(v, id);
            }
        }
    };

    for (int i = 0; i < n; i++) {
        if (comp_id[i] == -1) {
            dfs(i, comp_count);
            comp_count++;
        }
    }

    return {comp_id, comp_count};
}
```

```
// BFS-based version for grid (Number of Islands)
int num_islands(vector<vector<char>> &grid) {
    if (grid.empty() || grid[0].empty()) return 0;
    int rows = grid.size(), cols = grid[0].size();
    int count = 0;
    int dirs[4][2] = {{0,1},{0,-1},{1,0},{-1,0}};

    function<void(int,int)> dfs = [&](int r, int c) {
        if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] != '1')
            return;
        grid[r][c] = '0'; // mark visited
        for (auto &d : dirs)
            dfs(r + d[0], c + d[1]);
    };

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1') {
                count++;
                dfs(r, c);
            }
        }
    }
    return count;
}

int main() {
    int n = 5;
    vector<vector<int>> adj(n);
    adj[0] = {1};
    adj[1] = {0};
    adj[3] = {4};
    adj[4] = {3};

    auto [comp_id, comp_count] = connected_components(n, adj);
    cout << "Number of components: " << comp_count << "\n";
    for (int i = 0; i < n; i++)
        cout << "Node " << i << " -> component " << comp_id[i] << "\n";

    // Grid example
    vector<vector<char>> grid = {
        {'1','1','0','0','0'},
        {'1','1','0','0','0'},
        {'0','0','1','0','0'},
        {'0','0','0','1','1'}
    };
    cout << "Number of islands: " << num_islands(grid) << "\n";
    return 0;
}
```

9. Python Implementation

```
def connected_components(n, adj):
    comp_id = [-1] * n
    comp_count = 0

    def dfs(u, cid):
        comp_id[u] = cid
        for v in adj[u]:
            if comp_id[v] == -1:
                dfs(v, cid)

    for i in range(n):
        if comp_id[i] == -1:
            dfs(i, comp_count)
            comp_count += 1

    return comp_id, comp_count

def num_islands(grid):
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    dirs = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != '1':
            return
        grid[r][c] = '0'
        for dr, dc in dirs:
            dfs(r + dr, c + dc)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)

    return count

if __name__ == "__main__":
    n = 5
    adj = [[] for _ in range(n)]
    adj[0] = [1]
    adj[1] = [0]
    adj[3] = [4]
    adj[4] = [3]

    comp_id, comp_count = connected_components(n, adj)
    print(f"Number of components: {comp_count}")
```

```
for i in range(n):  
    print(f"Node {i} -> component {comp_id[i]}")
```

10. Code Explanation

- **DFS-based:** Recursive DFS assigns component ID to each visited node. The outer loop catches unvisited nodes and starts a new component.
- **Grid version:** Flips visited cells to '0' to mark them as visited (modifies input). Counts each DFS call as a new island.
- **Component ID array:** Maps each node to its component index. Useful for later processing.

11. Complexity Analysis

Operation	Time	Space
Find all components	$O(V + E)$	$O(V)$ — visited array + recursion stack
Grid islands	$O(\text{rows} \times \text{cols})$	$O(\text{rows} \times \text{cols})$ worst-case recursion

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Count components	"number of islands/provinces/groups"	Full DFS/BFS, count starts	LeetCode 200, 547
Largest component	"largest group", "biggest island"	Track size during DFS	LeetCode 695
Component labelling	"assign group id", "mark regions"	Store component ID per node	LeetCode 130

13. Common Mistakes

- **Not visiting all nodes** — forgetting the outer loop for disconnected graphs.
- **Modifying input permanently** — only do if allowed.
- **Using BFS/DFS incorrectly** — both work, but DFS is simpler for this.
- **Not resetting visited** between components — the algorithm handles this naturally.

14. Edge Cases

- Empty graph (0 components)
- Single node (1 component)
- Fully connected graph (1 component)
- No edges (each node is its own component — n components)
- Grid with no '1's (0 islands)

15. Variations

- **Strongly connected components** (SCC) — for directed graphs (Kosaraju, Tarjan)
- **Dynamic connectivity** — DSU handles edge additions efficiently
- **2D grid components** — special case of component finding on a grid

16. Practice Problems

Problem	Platform	Difficulty
200. Number of Islands	LeetCode	Easy
547. Number of Provinces	LeetCode	Easy
695. Max Area of Island	LeetCode	Medium
1905. Count Sub Islands	LeetCode	Medium

Cycle Detection in Undirected Graph

1. Overview

Detecting whether an undirected graph contains a cycle — a closed path where the first and last vertex are the same, and no edge is repeated.

2. Intuition

In an undirected graph, if during DFS you encounter a neighbour that is already visited **and it is not the parent** of the current node, you've found a cycle.

Think of walking through a maze: if you reach a room you've already visited, and you didn't just come from there, then there are two different paths to that room — forming a cycle.

Step-by-step reasoning:

1. Start DFS from any node.
2. Keep track of the parent (the node from which you came).
3. For each neighbour:
 - If neighbour is the parent, skip it.
 - If neighbour is visited (and not parent) → cycle exists.
 - If neighbour is unvisited, recurse.
4. If any recursive call finds a cycle, propagate **true**.

3. When to Use It

- Checking if a graph can be a tree (tree = connected + no cycles)
- Validating graph structures
- Detecting redundant edges
- **Trigger phrases:** "detect cycle", "is it a tree", "redundant connection", "valid tree"

4. When Not to Use It

- Directed graph — use DFS with recursion stack tracking (3-state visited) or Kahn's algorithm
- Need to list all cycles — that's a harder problem (finding **one** cycle is easy)
- Very large graph where recursion depth is a concern — use iterative DFS or DSU

5. Core Concepts

5.1 Parent

- The node from which the current node was discovered.
- **Why it matters:** In an undirected graph, the edge to parent will always be seen as "visited" — we must skip it to avoid false cycle detection.

5.2 Back Edge

- An edge connecting a node to an ancestor (other than parent) in the DFS tree.
- **Why it matters:** A back edge is exactly what constitutes a cycle in an undirected graph.

6. Step-by-Step Algorithm

1. Create `vis` array of size `n`, initialized to `false`.
2. For each unvisited node `u`:
 - Call `dfs(u, parent = -1)`.
3. In `dfs(u, parent)`:
 - Mark `vis[u] = true`.
 - For each neighbour `v` of `u`:
 - If `v == parent`, skip.
 - If `vis[v]` is true → cycle found, return `true`.
 - Else, call `dfs(v, u)`. If it returns `true`, propagate `true`.
 - Return `false` (no cycle in this component).

7. Dry Run

Graph: 0 — 1 — 2 — 0 (triangle)

Step	Call	Action	Visited	Stack
1	<code>dfs(0, -1)</code>	Mark 0, go to 1	{0}	[0]
2	<code>dfs(1, 0)</code>	Mark 1, go to 2	{0,1}	[0,1]
3	<code>dfs(2, 1)</code>	Mark 2, check neighbours: 0 is visited and not parent → cycle!	{0,1,2}	[0,1,2]
4	Return true	Propagate	—	—

Result: Cycle detected.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

bool dfs_cycle(int u, int parent, vector<vector<int>> &adj, vector<bool> &vis) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (v == parent) continue;
        if (vis[v]) return true; // back edge → cycle
        if (dfs_cycle(v, u, adj, vis)) return true;
    }
    return false;
}

bool has_cycle(int n, vector<vector<int>> &adj) {
    vector<bool> vis(n, false);
    for (int i = 0; i < n; i++) {
        if (!vis[i]) {
            if (dfs_cycle(i, -1, adj, vis)) return true;
        }
    }
    return false;
}

// DSU-based cycle detection (alternative)
struct DSU {
    vector<int> parent, rank;
    DSU(int n) : parent(n), rank(n, 0) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        return parent[x] == x ? x : (parent[x] = find(parent[x]));
    }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false; // cycle
        if (rank[a] < rank[b]) swap(a, b);
        parent[b] = a;
        if (rank[a] == rank[b]) rank[a]++;
        return true;
    }
};

bool has_cycle_dsu(int n, vector<vector<int>> &adj) {
    DSU dsu(n);
    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) {
            if (v < u) continue; // process each edge once
            if (!dsu.unite(u, v)) return true;
        }
    }
    return false;
}

int main() {
    int n = 3;
    vector<vector<int>> adj(n);

```

```

adj[0] = {1, 2};
adj[1] = {0, 2};
adj[2] = {0, 1};

cout << "Has cycle (DFS): " << (has_cycle(n, adj) ? "Yes" : "No") << "\n";
cout << "Has cycle (DSU): " << (has_cycle_dsu(n, adj) ? "Yes" : "No") << "\n";
return 0;
}

```

9. Python Implementation

```

def dfs_cycle(u, parent, adj, vis):
    vis[u] = True
    for v in adj[u]:
        if v == parent:
            continue
        if vis[v]:
            return True
        if dfs_cycle(v, u, adj, vis):
            return True
    return False

def has_cycle(n, adj):
    vis = [False] * n
    for i in range(n):
        if not vis[i]:
            if dfs_cycle(i, -1, adj, vis):
                return True
    return False

# DSU-based
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def unite(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return False
        if self.rank[a] < self.rank[b]:
            a, b = b, a
        self.parent[b] = a

```

```

        if self.rank[a] == self.rank[b]:
            self.rank[a] += 1
        return True

def has_cycle_dsu(n, adj):
    dsu = DSU(n)
    for u in range(n):
        for v in adj[u]:
            if v < u:
                continue
            if not dsu.unite(u, v):
                return True
    return False

if __name__ == "__main__":
    n = 3
    adj = [[1, 2], [0, 2], [0, 1]]
    print("Has cycle (DFS):", has_cycle(n, adj))
    print("Has cycle (DSU):", has_cycle_dsu(n, adj))

```

10. Code Explanation

- **DFS approach:** The `parent` parameter prevents going back along the same edge. If a visited neighbour (not parent) is found, there's a back edge → cycle.
- **DSU approach:** For each edge (u,v), if `u` and `v` are already in the same set, adding this edge creates a cycle. This is more efficient for static graphs.
- **Outer loop:** Handles disconnected graphs. Each component is checked independently.

11. Complexity Analysis

Method	Time	Space
DFS	$O(V + E)$	$O(V)$
DSU	$O(V + E \cdot \alpha(V))$	$O(V)$

12. Common Mistakes

- **Not passing parent** — causes false positives (every edge to parent looks like a cycle).
- **Forgetting disconnected graphs** — cycle may be in a different component.
- **Using DSU incorrectly** — must process each undirected edge only once.

13. Practice Problems

Problem	Platform	Difficulty
684. Redundant Connection	LeetCode	Medium
261. Graph Valid Tree	LeetCode	Medium

Problem	Platform	Difficulty
Detect cycle in an undirected graph	GFG	Easy

Cycle Detection in Directed Graph

1. Overview

Detecting a cycle in a **directed** graph. Since edges have direction, we need a different approach than undirected cycle detection.

2. Intuition

In a directed graph, a cycle exists if there is a path from a node back to itself following the direction of edges. We need to track nodes that are **currently in the recursion stack** (the current DFS path).

Think of it as following a series of one-way streets. If you ever return to an intersection that's already on your current route, you've found a cycle.

Step-by-step reasoning:

1. Each node has three states: unvisited (0), in current path (1), processed (2).
2. During DFS, when you visit a node, mark it as "in current path" (1).
3. For each neighbour:
 - If unvisited → recurse.
 - If in current path → cycle detected.
 - If processed → skip (already fully explored).
4. After processing all neighbours, mark node as "processed" (2).
5. Backtrack.

3. When to Use It

- Detecting deadlocks in dependency graphs
- Validating if a directed graph is a DAG (directed acyclic graph)
- Course schedule prerequisite validation
- **Trigger phrases:** "course schedule", "prerequisite cycle", "directed cycle", "DAG", "dependency graph"

4. When Not to Use It

- Undirected graph — simpler parent-based approach works
- Need topological order — Kahn's algorithm (BFS-based) also detects cycles and gives ordering
- Graph is very large — iterative approach may be safer

5. Core Concepts

5.1 Three-State Visited Array

- 0 = unvisited

- 1 = in current recursion stack (on current path)
- 2 = fully processed
- **Why it matters:** A node with state 2 has been completely explored; no cycle passes through it. A node with state 1 means we've found a back edge in the DFS tree.

5.2 Back Edge (Directed)

- An edge from a node to an ancestor in the DFS tree.
- **Why it matters:** In a directed graph, a back edge is the only type of edge that creates a cycle.

5.3 DAG (Directed Acyclic Graph)

- A directed graph with no cycles.
- **Why it matters:** DAGs can be topologically sorted. Many problems (scheduling, dependencies) require DAGs.

6. Step-by-Step Algorithm

1. Create `state` array of size `n`, initialized to 0 (unvisited).
2. For each unvisited node `u`:
 - Call `dfs(u)`.
3. In `dfs(u)`:
 - Set `state[u] = 1` (in current path).
 - For each neighbour `v`:
 - If `state[v] == 0` → recurse. If `true`, propagate.
 - If `state[v] == 1` → cycle found, return `true`.
 - If `state[v] == 2` → skip.
 - Set `state[u] = 2` (processed).
 - Return `false`.

7. Dry Run

Graph: $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$

Step	Call	State	Action
1	dfs(0)	state[0]=1	Visit 0, go to 1
2	dfs(1)	state[1]=1	Visit 1, go to 2
3	dfs(2)	state[2]=1	Visit 2, neighbour 0 has state=1 → cycle!
4	Return true	—	Propagate

Result: Cycle detected.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

bool dfs_cycle(int u, vector<vector<int>> &adj, vector<int> &state) {
    state[u] = 1; // in current path
    for (int v : adj[u]) {
        if (state[v] == 0) {
            if (dfs_cycle(v, adj, state)) return true;
        } else if (state[v] == 1) {
            return true; // back edge → cycle
        }
    }
    state[u] = 2; // processed
    return false;
}

bool has_cycle(int n, vector<vector<int>> &adj) {
    vector<int> state(n, 0); // 0=unvisited, 1=in stack, 2=done
    for (int i = 0; i < n; i++) {
        if (state[i] == 0) {
            if (dfs_cycle(i, adj, state)) return true;
        }
    }
    return false;
}

// Kahn's algorithm (BFS-based) for cycle detection + topological order
bool has_cycle_kahn(int n, vector<vector<int>> &adj) {
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; u++)
        for (int v : adj[u]) indeg[v]++;

    queue<int> q;
    for (int i = 0; i < n; i++)
        if (indeg[i] == 0) q.push(i);

    int count = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        count++;
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);
    }
    return count != n; // cycle if not all nodes processed
}

int main() {
    int n = 3;
    vector<vector<int>> adj(n);
    adj[0] = {1};
    adj[1] = {2};
    adj[2] = {0};

    cout << "Has cycle (DFS): " << (has_cycle(n, adj) ? "Yes" : "No") << "\n";
    cout << "Has cycle (Kahn): " << (has_cycle_kahn(n, adj) ? "Yes" : "No") <<
    "\n";
}

```

```

    return 0;
}

```

9. Python Implementation

```

def dfs_cycle(u, adj, state):
    state[u] = 1 # in current path
    for v in adj[u]:
        if state[v] == 0:
            if dfs_cycle(v, adj, state):
                return True
        elif state[v] == 1:
            return True
    state[u] = 2 # processed
    return False

def has_cycle(n, adj):
    state = [0] * n
    for i in range(n):
        if state[i] == 0:
            if dfs_cycle(i, adj, state):
                return True
    return False

def has_cycle_kahn(n, adj):
    indeg = [0] * n
    for u in range(n):
        for v in adj[u]:
            indeg[v] += 1

    q = deque([i for i in range(n) if indeg[i] == 0])
    count = 0

    while q:
        u = q.popleft()
        count += 1
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)

    return count != n

if __name__ == "__main__":
    n = 3
    adj = [[1], [2], [0]]

```

```
print("Has cycle (DFS):", has_cycle(n, adj))
print("Has cycle (Kahn):", has_cycle_kahn(n, adj))
```

10. Code Explanation

- **DFS with 3-state array:** The key difference from undirected cycle detection. State 1 means "currently on the DFS path". If we encounter a node with state 1, we've found a back edge.
- **Kahn's algorithm:** Uses in-degree. Nodes with indegree 0 have no dependencies. Process them, reduce indegrees of neighbours. If all nodes are processed, no cycle. If some remain, they form a cycle.
- **State 2 (processed):** Nodes that have been fully explored. No need to revisit them.

11. Complexity Analysis

Method	Time	Space
DFS (3-state)	$O(V + E)$	$O(V)$
Kahn's algorithm	$O(V + E)$	$O(V)$

12. Common Mistakes

- **Using only visited/unvisited (2 states)** — will miss cycles that are not reachable from the current DFS root.
- **Not resetting recursion stack** between components — each component needs fresh tracking.
- **Confusing undirected cycle detection** — parent check doesn't work for directed graphs.
- **Kahn's: forgetting to process all nodes with indegree 0 initially.**

13. Practice Problems

Problem	Platform	Difficulty
207. Course Schedule	LeetCode	Medium
210. Course Schedule II	LeetCode	Medium
Detect cycle in a directed graph	GFG	Medium

Bipartite Graph

1. Overview

A bipartite graph is a graph whose vertices can be divided into two disjoint sets (U and V) such that every edge connects a vertex in U to a vertex in V. No edge connects two vertices in the same set.

2. Intuition

Think of assigning **colours** (say red and blue) to nodes such that adjacent nodes always have different colours. If this is possible, the graph is bipartite.

Equivalently, a bipartite graph contains **no odd-length cycles**. Because an odd cycle forces two adjacent nodes to have the same colour.

Step-by-step reasoning:

1. Start with any node, colour it red (0).
2. BFS/DFS from it: colour all neighbours with the opposite colour (blue = 1).
3. If a neighbour already has a colour, check it matches the expected colour.
4. If any neighbour has the same colour → not bipartite.

3. When to Use It

- Checking if a graph can be split into two groups with no internal edges
- Matching problems (assignment, marriage, job allocation)
- Scheduling problems where certain pairs cannot be in the same group
- **Trigger phrases:** "bipartite", "two-colour", "can be divided into two groups", "odd cycle", "matching"

4. When Not to Use It

- Graph is guaranteed to be bipartite (tree, etc.) — no need to check
- Need maximum matching — use Hopcroft-Karp or Hungarian algorithm instead
- Graph has self-loops — self-loop makes it non-bipartite immediately

5. Core Concepts

5.1 Two-Colouring

- Assigning one of two colours to each vertex such that adjacent vertices have different colours.
- **Why it matters:** A graph is bipartite iff it is 2-colourable.

5.2 Odd Cycle Theorem

- A graph is bipartite iff it contains no odd-length cycle.
- **Why it matters:** Equivalent to the two-colouring definition.

5.3 Bipartite Matching

- Finding a maximum set of edges with no common vertices.
- **Why it matters:** Fundamental problem in combinatorial optimisation.

6. Step-by-Step Algorithm

1. Create `colour` array of size `n`, initialized to `-1` (uncoloured).
2. For each uncoloured node `u`:
 - Set `colour[u] = 0` (red).
 - Run BFS/DFS:
 - For each neighbour `v` of current node `x`:
 - If `colour[v] == -1`: set `colour[v] = 1 - colour[x]`.
 - Else if `colour[v] == colour[x]`: return `false` (not bipartite).
3. Return `true` (entire graph is bipartite).

7. Dry Run

Graph: $0 - 1 - 2 - 3 - 4$ (path of 5 nodes, no odd cycle $\rightarrow$ bipartite)

Step	Node	colour	Action
1	0	0	Start, colour 0 red
2	1	1	Neighbour of 0, colour blue
3	2	0	Neighbour of 1, colour red
4	3	1	Neighbour of 2, colour blue
5	4	0	Neighbour of 3, colour red

Result: Bipartite $\checkmark$

Graph with odd cycle: $0 - 1 - 2 - 0$

Step	Node	colour	Action
1	0	0	Start, colour 0 red
2	1	1	Neighbour of 0, colour blue
3	2	0	Neighbour of 1, colour red
4	0	0	From 2, neighbour 0 is red. Expected blue ($1 - 0 = 1$). Conflict!

Result: Not bipartite $\times$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// BFS-based bipartite check
bool is_bipartite_bfs(int n, vector<vector<int>> &adj) {
    vector<int> colour(n, -1);

    for (int start = 0; start < n; start++) {
        if (colour[start] != -1) continue;

        colour[start] = 0;
        queue<int> q;
        q.push(start);

        while (!q.empty()) {
            int u = q.front();
            q.pop();

            for (int v : adj[u]) {
                if (colour[v] == -1) {
```

```

        colour[v] = 1 - colour[u];
        q.push(v);
    } else if (colour[v] == colour[u]) {
        return false;
    }
}
}
}
return true;
}

// DFS-based bipartite check
bool dfs_bipartite(int u, int c, vector<vector<int>> &adj, vector<int> &colour) {
    colour[u] = c;
    for (int v : adj[u]) {
        if (colour[v] == -1) {
            if (!dfs_bipartite(v, 1 - c, adj, colour)) return false;
        } else if (colour[v] == colour[u]) {
            return false;
        }
    }
    return true;
}

bool is_bipartite_dfs(int n, vector<vector<int>> &adj) {
    vector<int> colour(n, -1);
    for (int i = 0; i < n; i++) {
        if (colour[i] == -1) {
            if (!dfs_bipartite(i, 0, adj, colour)) return false;
        }
    }
    return true;
}

int main() {
    int n = 4;
    vector<vector<int>> adj(n);
    adj[0] = {1, 3};
    adj[1] = {0, 2};
    adj[2] = {1, 3};
    adj[3] = {0, 2}; // even cycle → bipartite

    cout << "Bipartite: " << (is_bipartite_bfs(n, adj) ? "Yes" : "No") << "\n";
    return 0;
}

```

9. Python Implementation

```

from collections import deque

def is_bipartite_bfs(n, adj):

```

```

colour = [-1] * n

for start in range(n):
    if colour[start] != -1:
        continue

    colour[start] = 0
    q = deque([start])

    while q:
        u = q.popleft()
        for v in adj[u]:
            if colour[v] == -1:
                colour[v] = 1 - colour[u]
                q.append(v)
            elif colour[v] == colour[u]:
                return False
    return True

def dfs_bipartite(u, c, adj, colour):
    colour[u] = c
    for v in adj[u]:
        if colour[v] == -1:
            if not dfs_bipartite(v, 1 - c, adj, colour):
                return False
        elif colour[v] == colour[u]:
            return False
    return True

def is_bipartite_dfs(n, adj):
    colour = [-1] * n
    for i in range(n):
        if colour[i] == -1:
            if not dfs_bipartite(i, 0, adj, colour):
                return False
    return True

if __name__ == "__main__":
    n = 4
    adj = [[1, 3], [0, 2], [1, 3], [0, 2]]
    print("Bipartite:", is_bipartite_bfs(n, adj))

```

10. Code Explanation

- **BFS approach:** Level-by-level colouring. Since BFS gives levels, all nodes at the same level should have the same colour. If a neighbour has the same colour as the current node, it's not bipartite.
- **DFS approach:** Recursively colour and check. Simple and elegant.
- **1 - colour[u] trick:** Flips between 0 and 1.
- **Outer loop:** Handles disconnected graphs.

11. Complexity Analysis

Operation	Time	Space
BFS/DFS bipartite check	$O(V + E)$	$O(V)$

12. Common Mistakes

- **Not handling disconnected components** — each component must be checked independently.
- **Using only 2 states (visited/unvisited)** — need colour array.
- **Assuming BFS and DFS give same result** — they do, but implementation differs.
- **Not checking all neighbours** — must check all neighbours for conflict.

13. Practice Problems

Problem	Platform	Difficulty
785. Is Graph Bipartite?	LeetCode	Medium
886. Possible Bipartition	LeetCode	Medium
CSES Building Teams	CSES	Easy

Flood Fill

1. Overview

Flood Fill is an algorithm that determines the area connected to a given node in a multi-dimensional array. It's the algorithm behind the "paint bucket" tool in graphics editors.

2. Intuition

Think of pouring paint on a canvas. The paint spreads in all directions until it hits a boundary of a different colour. Everything with the same original colour that is connected to the starting point gets filled with the new colour.

Step-by-step reasoning:

1. Start at the given pixel.
2. If the pixel's colour is not the target colour, return.
3. Change the pixel's colour to the new colour.
4. Recursively/iteratively do the same for all 4-directional neighbours.

3. When to Use It

- Image/pixel fill operations
- Connected component labelling in grids
- Region filling in mazes
- **Trigger phrases:** "flood fill", "paint bucket", "fill connected region", "change all adjacent same colour"

4. When Not to Use It

- Very large grid (recursion may overflow) — use iterative BFS/DFS
- Need to preserve original colours — make a copy first
- Grid is extremely sparse — other approaches may be better

5. Core Concepts

5.1 4-Direction vs 8-Direction Connectivity

- 4-direction: up, down, left, right
- 8-direction: includes diagonals
- **Why it matters:** Changes the flood fill region.

5.2 Boundary Condition

- Stopping when pixel colour is different from the original colour.
- **Why it matters:** Defines the region boundary.

6. Step-by-Step Algorithm

1. If `grid[sr][sc] == newColour`, return (no change needed).
2. Store `originalColour = grid[sr][sc]`.
3. Call DFS/BFS from `(sr, sc)`.
4. In DFS:
 - If out of bounds, return.
 - If `grid[r][c] != originalColour`, return.
 - Set `grid[r][c] = newColour`.
 - Recurse on 4 neighbours.

7. Dry Run

Grid:

```
1 1 0
1 0 0
0 0 1
```

Start: (0,0), newColour = 2, originalColour = 1

Step	Position	Action	Grid
Start	(0,0)	original=1, change to 2	2 1 0 / 1 0 0 / 0 0 1
1	(0,1)	original=1, change to 2	2 2 0 / 1 0 0 / 0 0 1
2	(1,0)	original=1, change to 2	2 2 0 / 2 0 0 / 0 0 1
3	(1,1)	original=0, skip	2 2 0 / 2 0 0 / 0 0 1

Step	Position	Action	Grid
4	(0,-1)	out of bounds	—
5	(-1,0)	out of bounds	—
6	(0,2)	original=0, skip	—
7	(2,0)	original=0, skip	—

Result: All 1s connected to (0,0) become 2.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// DFS-based flood fill
void flood_fill_dfs(vector<vector<int>> &grid, int r, int c,
                    int original, int newColour) {
    int rows = grid.size(), cols = grid[0].size();
    if (r < 0 || r >= rows || c < 0 || c >= cols) return;
    if (grid[r][c] != original) return;

    grid[r][c] = newColour;
    flood_fill_dfs(grid, r + 1, c, original, newColour);
    flood_fill_dfs(grid, r - 1, c, original, newColour);
    flood_fill_dfs(grid, r, c + 1, original, newColour);
    flood_fill_dfs(grid, r, c - 1, original, newColour);
}

vector<vector<int>> flood_fill(vector<vector<int>> &grid, int sr, int sc, int
newColour) {
    if (grid[sr][sc] == newColour) return grid;
    flood_fill_dfs(grid, sr, sc, grid[sr][sc], newColour);
    return grid;
}

// BFS-based flood fill (iterative, safe for large grids)
vector<vector<int>> flood_fill_bfs(vector<vector<int>> &grid, int sr, int sc, int
newColour) {
    int rows = grid.size(), cols = grid[0].size();
    int original = grid[sr][sc];
    if (original == newColour) return grid;

    int dirs[4][2] = {{0,1},{0,-1},{1,0},{-1,0}};
    queue<pair<int,int>> q;
    q.push({sr, sc});
    grid[sr][sc] = newColour;

    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        for (auto &d : dirs) {
```

```

        int nr = r + d[0], nc = c + d[1];
        if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] ==
original) {
            grid[nr][nc] = newColour;
            q.push({nr, nc});
        }
    }
}
return grid;
}

int main() {
    vector<vector<int>> grid = {{1,1,0},{1,0,0},{0,0,1}};
    flood_fill(grid, 0, 0, 2);
    for (auto &row : grid) {
        for (int x : row) cout << x << " ";
        cout << "\n";
    }
    return 0;
}

```

9. Python Implementation

```

def flood_fill_dfs(grid, r, c, original, new_colour):
    rows, cols = len(grid), len(grid[0])
    if r < 0 or r >= rows or c < 0 or c >= cols:
        return
    if grid[r][c] != original:
        return

    grid[r][c] = new_colour
    flood_fill_dfs(grid, r + 1, c, original, new_colour)
    flood_fill_dfs(grid, r - 1, c, original, new_colour)
    flood_fill_dfs(grid, r, c + 1, original, new_colour)
    flood_fill_dfs(grid, r, c - 1, original, new_colour)

def flood_fill(grid, sr, sc, new_colour):
    if grid[sr][sc] == new_colour:
        return grid
    flood_fill_dfs(grid, sr, sc, grid[sr][sc], new_colour)
    return grid

def flood_fill_bfs(grid, sr, sc, new_colour):
    rows, cols = len(grid), len(grid[0])
    original = grid[sr][sc]
    if original == new_colour:
        return grid

    dirs = [(0, 1), (0, -1), (1, 0), (-1, 0)]

```

```

q = deque([(sr, sc)])
grid[sr][sc] = new_colour

while q:
    r, c = q.popleft()
    for dr, dc in dirs:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == original:
            grid[nr][nc] = new_colour
            q.append((nr, nc))

return grid

if __name__ == "__main__":
    grid = [[1, 1, 0], [1, 0, 0], [0, 0, 1]]
    flood_fill(grid, 0, 0, 2)
    for row in grid:
        print(row)

```

10. Code Explanation

- **DFS version:** Recursive, simple. Risk of stack overflow on large grids.
- **BFS version:** Iterative, safe for large grids. Uses a queue.
- **Early return:** If `original == newColour`, return immediately to avoid infinite loops.
- **4-directional:** Standard. 8-directional requires adding diagonals.

11. Complexity Analysis

Operation	Time	Space
Flood fill (DFS/BFS)	$O(\text{rows} \times \text{cols})$	$O(\text{rows} \times \text{cols})$ worst-case recursion stack / queue

12. Practice Problems

Problem	Platform	Difficulty
733. Flood Fill	LeetCode	Easy
200. Number of Islands	LeetCode	Medium

Matrix BFS/DFS

1. Overview

Running BFS/DFS on a 2D grid (matrix) is a common pattern. Each cell is a node, and edges connect adjacent cells (4-directional or 8-directional).

2. Intuition

Think of a grid as a graph where each cell is a vertex and edges exist between orthogonal neighbours. BFS on a grid gives shortest path in terms of steps. DFS on a grid explores all paths.

Step-by-step reasoning:

1. Represent each cell by its coordinates (r, c) .
2. Use direction arrays: `dirs = {{0,1},{0,-1},{1,0},{-1,0}}`.
3. For BFS, use a queue. For DFS, use recursion or stack.
4. Check bounds and visited/wall conditions before visiting.

3. When to Use It

- Shortest path in a grid (BFS)
- Counting connected regions (DFS/BFS)
- Maze solving
- **Trigger phrases:** "grid", "matrix", "maze", "shortest path in grid", "island", "obstacle"

4. Core Concepts

4.1 Direction Arrays

- `dr = {0, 0, 1, -1}, dc = {1, -1, 0, 0}` for 4-directional
- `dr = {0,0,1,-1,1,1,-1,-1}, dc = {1,-1,0,0,1,-1,1,-1}` for 8-directional
- **Why it matters:** Clean, reusable way to iterate over neighbours.

4.2 Visited Array

- A separate 2D boolean array, or modify the grid in-place.
- **Why it matters:** Prevents revisiting cells.

4.3 Bounds Check

- `r >= 0 && r < rows && c >= 0 && c < cols.`
- **Why it matters:** Prevents out-of-bounds access.

5. Step-by-Step Algorithm (BFS on Grid)

1. Create `dist` 2D array, initialize to `-1` or `INF`.
2. Create queue, push source (sr, sc) , set `dist[sr][sc] = 0`.
3. While queue not empty:
 - Pop (r, c) .
 - For each of 4 directions:
 - Compute `nr = r + dr[i], nc = c + dc[i]`.
 - If in bounds, not visited, and not obstacle:
 - Set `dist[nr][nc] = dist[r][c] + 1`.
 - Push (nr, nc) .
4. Return `dist` (or distance to target).

6. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// BFS shortest path in grid (0 = empty, 1 = obstacle)
int bfs_grid(vector<vector<int>> &grid, pair<int,int> src, pair<int,int> dest) {
    int rows = grid.size(), cols = grid[0].size();
    vector<vector<int>> dist(rows, vector<int>(cols, -1));
    int dirs[4][2] = {{0,1},{0,-1},{1,0},{-1,0}};

    queue<pair<int,int>> q;
    q.push(src);
    dist[src.first][src.second] = 0;

    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();

        if (r == dest.first && c == dest.second)
            return dist[r][c];

        for (auto &d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
                && grid[nr][nc] == 0 && dist[nr][nc] == -1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }
    return -1; // unreachable
}

// DFS on grid (count connected cells of value 1)
int dfs_grid(vector<vector<int>> &grid, int r, int c) {
    int rows = grid.size(), cols = grid[0].size();
    if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] != 1)
        return 0;

    grid[r][c] = 0; // mark visited
    int count = 1;
    int dirs[4][2] = {{0,1},{0,-1},{1,0},{-1,0}};
    for (auto &d : dirs)
        count += dfs_grid(grid, r + d[0], c + d[1]);
    return count;
}

int main() {
    vector<vector<int>> grid = {
        {0,0,1,0},
        {0,0,1,0},
        {0,0,0,0},
        {1,1,0,0}
    };
}

```

```

int dist = bfs_grid(grid, {0,0}, {3,3});
cout << "Shortest distance: " << dist << "\n"; // 6

vector<vector<int>> grid2 = {
    {1,1,0,0},
    {1,1,0,0},
    {0,0,1,0},
    {0,0,0,1}
};
int area = dfs_grid(grid2, 0, 0);
cout << "Area of component: " << area << "\n"; // 4
return 0;
}

```

7. Python Implementation

```

from collections import deque

def bfs_grid(grid, src, dest):
    rows, cols = len(grid), len(grid[0])
    dist = [[-1] * cols for _ in range(rows)]
    dirs = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    q = deque([src])
    dist[src[0]][src[1]] = 0

    while q:
        r, c = q.popleft()
        if (r, c) == dest:
            return dist[r][c]

        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 0 and
dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))

    return -1

def dfs_grid(grid, r, c):
    rows, cols = len(grid), len(grid[0])
    if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != 1:
        return 0

    grid[r][c] = 0
    count = 1
    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]:

```

```
count += dfs_grid(grid, r + dr, c + dc)
return count
```

8. Complexity Analysis

Operation	Time	Space
BFS on grid	$O(\text{rows} \times \text{cols})$	$O(\text{rows} \times \text{cols})$
DFS on grid	$O(\text{rows} \times \text{cols})$	$O(\text{rows} \times \text{cols})$ worst-case recursion

9. Common Mistakes

- **Forgetting bounds check** — accessing `grid[r][c]` when `r` or `c` is out of range.
- **Using `grid[r][c] == 0` for both empty and visited** — confusion between obstacle (0) and visited (0).
- **Not using direction arrays** — leads to repetitive code and bugs.
- **Modifying grid in-place when not allowed** — ensure you're allowed to modify input.

10. Practice Problems

Problem	Platform	Difficulty
1091. Shortest Path in Binary Matrix	LeetCode	Medium
417. Pacific Atlantic Water Flow	LeetCode	Medium
1293. Shortest Path in a Grid with Obstacles Elimination	LeetCode	Hard

Multi-Source BFS

1. Overview

Multi-source BFS starts BFS from **multiple source nodes** simultaneously. All sources are pushed into the queue at the start, and BFS proceeds level by level from all of them.

2. Intuition

Think of multiple fires starting at different points and spreading outward simultaneously. The time when a location catches fire is the minimum distance from that location to any fire source.

Step-by-step reasoning:

1. Push all source nodes into the queue with distance 0.
2. Run BFS normally.
3. The distance computed for each node is the minimum distance to **any** source.

3. When to Use It

- Finding nearest distance to any of a set of targets
- Problems with multiple starting points spreading simultaneously
- **Trigger phrases:** "nearest 0", "distance to nearest", "rotten oranges", "all sources", "minimum distance to any"

4. Core Concepts

4.1 Multiple Sources

- All sources start at distance 0 and are pushed into the queue initially.
- **Why it matters:** BFS processes them level by level, giving min distance to the nearest source.

4.2 Simultaneous Propagation

- The wavefront from all sources moves outward together.
- **Why it matters:** Guarantees shortest distance to the nearest source.

5. Step-by-Step Algorithm

1. Create `dist` 2D array, initialize to `-1`.
2. For each source `(r, c)`:
 - Set `dist[r][c] = 0`.
 - Push `(r, c)` into queue.
3. While queue not empty:
 - Pop `(r, c)`.
 - For each neighbour:
 - If `dist[nr][nc] == -1`:
 - Set `dist[nr][nc] = dist[r][c] + 1`.
 - Push `(nr, nc)`.
4. Return `dist`.

6. Dry Run

Grid:

```
0 0 0
0 1 0
1 1 1
```

Source: all 1s. Find distance of each cell to nearest 1.

Step	Queue	Distances
Init	(1,1),(2,0),(2,1),(2,2)	$\infty \infty \infty / \infty 0 \infty / 0 0 0$
Pop (1,1) → push (0,1),(1,0),(1,2),(2,1)	(2,0),(2,1),(2,2),(0,1),(1,0),(1,2)	$\infty 1 \infty / 1 0 1 / 0 0 0$
Pop (2,0) → push (1,0) already visited	...	$1 1 2 / 1 0 1 / 0 0 0$

Step	Queue	Distances
...	...	...

Final distances:

```
1 1 2
1 0 1
0 0 0
```

7. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Multi-source BFS: for each cell, find distance to nearest 1
vector<vector<int>> multi_source_bfs(vector<vector<int>> &grid) {
    int rows = grid.size(), cols = grid[0].size();
    vector<vector<int>> dist(rows, vector<int>(cols, -1));
    queue<pair<int,int>> q;
    int dirs[4][2] = {{0,1},{0,-1},{1,0},{-1,0}};

    // Push all sources
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 1) {
                dist[r][c] = 0;
                q.push({r, c});
            }
        }
    }

    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        for (auto &d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && dist[nr][nc] ==
-1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }
    return dist;
}

// Rotten Oranges problem
int orangesRotting(vector<vector<int>> &grid) {
    int rows = grid.size(), cols = grid[0].size();
    queue<pair<int,int>> q;
```

```

int fresh = 0, minutes = 0;
int dirs[4][2] = {{0,1},{0,-1},{1,0},{-1,0}};

for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        if (grid[r][c] == 2) q.push({r, c});
        else if (grid[r][c] == 1) fresh++;
    }
}

while (!q.empty() && fresh > 0) {
    int sz = q.size();
    minutes++;
    while (sz--) {
        auto [r, c] = q.front(); q.pop();
        for (auto &d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc]
== 1) {
                grid[nr][nc] = 2;
                fresh--;
                q.push({nr, nc});
            }
        }
    }
}
return fresh == 0 ? minutes : -1;
}

int main() {
    vector<vector<int>> grid = {{0,0,0},{0,1,0},{1,1,1}};
    auto dist = multi_source_bfs(grid);
    for (auto &row : dist) {
        for (int x : row) cout << x << " ";
        cout << "\n";
    }
    // Output:
    // 1 1 2
    // 1 0 1
    // 0 0 0
    return 0;
}

```

8. Python Implementation

```

from collections import deque

def multi_source_bfs(grid):
    rows, cols = len(grid), len(grid[0])
    dist = [[-1] * cols for _ in range(rows)]
    q = deque()

```

```

dirs = [(0, 1), (0, -1), (1, 0), (-1, 0)]

for r in range(rows):
    for c in range(cols):
        if grid[r][c] == 1:
            dist[r][c] = 0
            q.append((r, c))

while q:
    r, c = q.popleft()
    for dr, dc in dirs:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols and dist[nr][nc] == -1:
            dist[nr][nc] = dist[r][c] + 1
            q.append((nr, nc))

return dist

def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    minutes = 0
    dirs = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                q.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1

    while q and fresh > 0:
        minutes += 1
        for _ in range(len(q)):
            r, c = q.popleft()
            for dr, dc in dirs:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))

    return minutes if fresh == 0 else -1

```

9. Complexity Analysis

Operation	Time	Space
Multi-source BFS	$O(\text{rows} \times \text{cols})$	$O(\text{rows} \times \text{cols})$

10. Common Mistakes

- **Starting BFS from only one source** — defeats the purpose.
- **Not using level-by-level processing** for time-based problems (like rotting oranges).
- **Forgetting to handle empty grid** or no sources case.

11. Practice Problems

Problem	Platform	Difficulty
542. 01 Matrix	LeetCode	Medium
994. Rotting Oranges	LeetCode	Medium
1162. As Far from Land as Possible	LeetCode	Medium

0-1 BFS

1. Overview

0-1 BFS finds shortest paths in a graph where edge weights are either **0 or 1**. It uses a **deque** (double-ended queue) instead of a regular queue, pushing 0-weight edges to the front and 1-weight edges to the back.

2. Intuition

Think of a maze where some paths are free (0 cost) and others cost 1. You want to minimise total cost. When you find a 0-weight edge, you want to process it immediately (it costs nothing extra, so it's like staying at the same level). When you find a 1-weight edge, you process it later (it's like moving to the next level).

Step-by-step reasoning:

1. Use a deque instead of a queue.
2. Push source to the front with distance 0.
3. While deque not empty:
 - Pop front node **u**.
 - For each neighbour **v** with weight **w** (0 or 1):
 - If `dist[u] + w < dist[v]`:
 - Update `dist[v] = dist[u] + w`.
 - If `w == 0`: push **v** to **front** of deque.
 - If `w == 1`: push **v** to **back** of deque.

Why it works: The deque maintains the invariant that distances are non-decreasing from front to back. 0-weight edges don't increase distance, so the neighbour is processed immediately (front). 1-weight edges increase distance, so the neighbour goes to the back.

3. When to Use It

- Shortest path in graphs with edge weights 0 and 1 only
- Grid with free cells (0) and cost-1 cells (1)

- Minimising number of "special" operations
- **Trigger phrases:** "0 and 1 weights", "minimum cost with two types of moves", "free moves and paid moves"

4. When Not to Use It

- Edge weights other than 0 or 1 — use Dijkstra
- Equal weights — use standard BFS
- Single source shortest path is overkill — BFS is simpler

5. Complexity Analysis

Operation	Time	Space
0-1 BFS	$O(V + E)$	$O(V)$

This is **faster than Dijkstra** ($O(V + E \log V)$) for the special case of 0-1 weights.

6. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// 0-1 BFS on a graph with edge weights 0 or 1
vector<int> zero_one_bfs(int n, vector<vector<pair<int,int>>> &adj, int source) {
    vector<int> dist(n, INT_MAX);
    deque<int> dq;

    dist[source] = 0;
    dq.push_front(source);

    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();

        for (auto &[v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0)
                    dq.push_front(v);
                else
                    dq.push_back(v);
            }
        }
    }

    return dist;
}

// 0-1 BFS on grid: 0 = free cell, 1 = cost-1 cell
int zero_one_bfs_grid(vector<vector<int>> &grid, pair<int,int> src, pair<int,int>
dest) {
```

```

int rows = grid.size(), cols = grid[0].size();
vector<vector<int>> dist(rows, vector<int>(cols, INT_MAX));
deque<pair<int,int>> dq;
int dirs[4][2] = {{0,1},{0,-1},{1,0},{-1,0}};

dist[src.first][src.second] = 0;
dq.push_front(src);

while (!dq.empty()) {
    auto [r, c] = dq.front(); dq.pop_front();

    if (r == dest.first && c == dest.second) return dist[r][c];

    for (auto &d : dirs) {
        int nr = r + d[0], nc = c + d[1];
        if (nr >= 0 && nr < rows && nc >= 0 && nc < cols) {
            int w = grid[nr][nc]; // 0 or 1
            if (dist[r][c] + w < dist[nr][nc]) {
                dist[nr][nc] = dist[r][c] + w;
                if (w == 0)
                    dq.push_front({nr, nc});
                else
                    dq.push_back({nr, nc});
            }
        }
    }
}
return -1;
}

int main() {
    // Graph example
    int n = 4;
    vector<vector<pair<int,int>>> adj(n);
    adj[0] = {{1, 0}, {2, 1}};
    adj[1] = {{2, 0}, {3, 1}};
    adj[2] = {{3, 0}};

    auto dist = zero_one_bfs(n, adj, 0);
    for (int i = 0; i < n; i++)
        cout << "dist[0->" << i << "] = " << dist[i] << "\n";

    // Grid example
    vector<vector<int>> grid = {
        {0, 0, 1},
        {1, 0, 0},
        {1, 1, 0}
    };
    int d = zero_one_bfs_grid(grid, {0,0}, {2,2});
    cout << "Shortest path in grid: " << d << "\n";
    return 0;
}

```

7. Python Implementation

```

from collections import deque

def zero_one_bfs(n, adj, source):
    dist = [float('inf')] * n
    dq = deque()

    dist[source] = 0
    dq.appendleft(source)

    while dq:
        u = dq.popleft()
        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return dist

def zero_one_bfs_grid(grid, src, dest):
    rows, cols = len(grid), len(grid[0])
    dist = [[float('inf')] * cols for _ in range(rows)]
    dq = deque()
    dirs = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    dist[src[0]][src[1]] = 0
    dq.appendleft(src)

    while dq:
        r, c = dq.popleft()
        if (r, c) == dest:
            return dist[r][c]

        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                w = grid[nr][nc]
                if dist[r][c] + w < dist[nr][nc]:
                    dist[nr][nc] = dist[r][c] + w
                    if w == 0:
                        dq.appendleft((nr, nc))
                    else:
                        dq.append((nr, nc))

    return -1

```

8. Common Mistakes

- **Using a regular queue** — loses the 0-1 optimisation.
- **Pushing 0-weight edges to the back** — violates the distance invariant.
- **Not checking for `dist[v]` update** — must relax edges like Dijkstra.
- **Confusing with BFS** — BFS is for uniform weights, 0-1 BFS is for two distinct weights.

9. Practice Problems

Problem	Platform	Difficulty
1368. Minimum Cost to Make at Least One Valid Path in a Grid	LeetCode	Hard
Minimum Cost to Reach Destination	SPOJ	Medium
CSES Labyrinth	CSES	Hard

DFS Timestamps

1. Overview

DFS timestamps (`tin` and `tout`) record the time when a node is first discovered and when it is fully processed (all its descendants are visited). These timestamps are fundamental for advanced graph algorithms.

2. Intuition

Think of a clock that ticks once per node visit. When you first enter a node, you record `tin[u] = time`. When you finish processing all its descendants and leave, you record `tout[u] = time`. This creates a time interval `[tin[u], tout[u]]` for each node.

Key property: For any two nodes `u` and `v`, the intervals are either disjoint or one contains the other. If `u` is an ancestor of `v` in the DFS tree, then `tin[u] < tin[v] < tout[v] < tout[u]`.

3. When to Use It

- Bridges (Tarjan's algorithm)
- Articulation points (Tarjan's algorithm)
- Strongly connected components (Tarjan's algorithm)
- Determining ancestor-descendant relationships in $O(1)$
- **Trigger phrases:** "tin", "tout", "dfs order", "ancestor check", "bridges", "articulation points"

4. Core Concepts

4.1 `tin[u]` — Entry Time

- The time when node `u` is first discovered.
- **Why it matters:** Used to determine if a neighbour is an ancestor or descendant.

4.2 `tout[u]` — Exit Time

- The time when node `u` and all its descendants are fully processed.

- **Why it matters:** $\text{tin}[u] < \text{tin}[v] < \text{tout}[v] < \text{tout}[u]$ means u is an ancestor of v .

4.3 Ancestor Check

- u is ancestor of v iff $\text{tin}[u] \leq \text{tin}[v] \ \&\& \ \text{tout}[v] \leq \text{tout}[u]$.
- **Why it matters:** $O(1)$ ancestor check, useful in many tree/graph problems.

5. Step-by-Step Algorithm

1. Global variable `timer = 0`.
2. In `DFS(u)`:
 - Set `tin[u] = timer++`.
 - For each neighbour v :
 - If not visited, call `dfs(v)`.
 - Set `tout[u] = timer++`.

6. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void dfs_timestamps(int u, vector<vector<int>> &adj, vector<bool> &vis,
                    vector<int> &tin, vector<int> &tout, int &timer) {
    vis[u] = true;
    tin[u] = timer++;

    for (int v : adj[u]) {
        if (!vis[v]) {
            dfs_timestamps(v, adj, vis, tin, tout, timer);
        }
    }

    tout[u] = timer++;
}

// Is u an ancestor of v?
bool is_ancestor(int u, int v, vector<int> &tin, vector<int> &tout) {
    return tin[u] <= tin[v] && tout[v] <= tout[u];
}

int main() {
    int n = 6;
    vector<vector<int>> adj(n);
    adj[0] = {1, 2};
    adj[1] = {0, 3, 4};
    adj[2] = {0, 5};
    adj[3] = {1};
    adj[4] = {1};
    adj[5] = {2};

    vector<bool> vis(n, false);
```

```

vector<int> tin(n), tout(n);
int timer = 0;

dfs_timestamps(0, adj, vis, tin, tout, timer);

for (int i = 0; i < n; i++) {
    cout << "Node " << i << ": tin=" << tin[i] << ", tout=" << tout[i] <<
"\n";
}

cout << "Is 0 ancestor of 3? " << (is_ancestor(0, 3, tin, tout) ? "Yes" :
"No") << "\n";
cout << "Is 3 ancestor of 0? " << (is_ancestor(3, 0, tin, tout) ? "Yes" :
"No") << "\n";
return 0;
}

```

7. Python Implementation

```

def dfs_timestamps(u, adj, vis, tin, tout, timer):
    vis[u] = True
    tin[u] = timer[0]
    timer[0] += 1

    for v in adj[u]:
        if not vis[v]:
            dfs_timestamps(v, adj, vis, tin, tout, timer)

    tout[u] = timer[0]
    timer[0] += 1

def is_ancestor(u, v, tin, tout):
    return tin[u] <= tin[v] <= tout[v] <= tout[u]

if __name__ == "__main__":
    n = 6
    adj = [[] for _ in range(n)]
    adj[0] = [1, 2]
    adj[1] = [0, 3, 4]
    adj[2] = [0, 5]
    adj[3] = [1]
    adj[4] = [1]
    adj[5] = [2]

    vis = [False] * n
    tin = [0] * n
    tout = [0] * n
    timer = [0]

```

```

dfs_timestamps(0, adj, vis, tin, tout, timer)

for i in range(n):
    print(f"Node {i}: tin={tin[i]}, tout={tout[i]}")

print(f"Is 0 ancestor of 3? {is_ancestor(0, 3, tin, tout)}")
print(f"Is 3 ancestor of 0? {is_ancestor(3, 0, tin, tout)}")

```

8. Complexity Analysis

Operation	Time	Space
Computing timestamps	$O(V + E)$	$O(V)$
Ancestor check	$O(1)$	$O(1)$

9. Practice Problems

Problem	Platform	Difficulty
1192. Critical Connections in a Network	LeetCode	Hard
1489. Find Critical and Pseudo-Critical Edges	LeetCode	Hard

Bridges

1. Overview

A **bridge** is an edge in a graph whose removal increases the number of connected components. Bridges are critical edges — if they break, the graph becomes disconnected.

2. Intuition

Think of a bridge in a network of islands. If you remove that single connection, two parts of the network become isolated. Bridges are "weak links" in the graph.

Tarjan's algorithm uses DFS timestamps to find bridges in $O(V + E)$ time.

Key insight: An edge (u, v) is a bridge if there is **no other path** from u to v (or from v 's subtree back to u or above) besides the edge itself. We check this using the **lowest** reachable ancestor.

3. When to Use It

- Finding critical connections in a network
- Identifying single points of failure
- Network reliability analysis
- Trigger phrases:** "critical connections", "bridges", "cut edge", "redundant connection"

4. Core Concepts

4.1 low[u] — Lowest Reachable Ancestor

- The minimum `tin` value reachable from `u` or its descendants by traversing at most one back edge.
- Why it matters:** If `low[v] > tin[u]`, then there's no back edge from subtree of `v` to `u` or above → edge `(u,v)` is a bridge.

4.2 Bridge Condition

- Edge `(u, v)` where `u` is parent of `v` in DFS tree is a bridge iff `low[v] > tin[u]`.
- Why it matters:** Simple condition derived from DFS timestamps.

5. Step-by-Step Algorithm

- Do DFS from any node.
- Maintain `tin[u]` (entry time) and `low[u]` (lowest reachable ancestor).
- When visiting neighbour `v` from `u`:
 - If `v` is not visited:
 - Recurse.
 - After returning, update `low[u] = min(low[u], low[v])`.
 - If `low[v] > tin[u]`, then `(u, v)` is a bridge.
 - Else if `v` is not parent:
 - `low[u] = min(low[u], tin[v])` (back edge).

6. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void find_bridges(int u, int parent, vector<vector<int>> &adj,
                 vector<bool> &vis, vector<int> &tin, vector<int> &low,
                 int &timer, vector<pair<int,int>> &bridges) {
    vis[u] = true;
    tin[u] = low[u] = timer++;

    for (int v : adj[u]) {
        if (v == parent) continue;

        if (!vis[v]) {
            find_bridges(v, u, adj, vis, tin, low, timer, bridges);
            low[u] = min(low[u], low[v]);

            // Bridge condition
            if (low[v] > tin[u]) {
                bridges.push_back({u, v});
            }
        } else {
            // Back edge
            low[u] = min(low[u], tin[v]);
        }
    }
}
```

```

}

vector<pair<int,int>> get_bridges(int n, vector<vector<int>> &adj) {
    vector<bool> vis(n, false);
    vector<int> tin(n, -1), low(n, -1);
    vector<pair<int,int>> bridges;
    int timer = 0;

    for (int i = 0; i < n; i++) {
        if (!vis[i]) {
            find_bridges(i, -1, adj, vis, tin, low, timer, bridges);
        }
    }
    return bridges;
}

int main() {
    int n = 5;
    vector<vector<int>> adj(n);
    adj[0] = {1, 2};
    adj[1] = {0, 2, 3};
    adj[2] = {0, 1};
    adj[3] = {1, 4};
    adj[4] = {3};

    auto bridges = get_bridges(n, adj);
    cout << "Bridges:\n";
    for (auto &[u, v] : bridges) {
        cout << u << " - " << v << "\n";
    }
    // Output: 1 - 3, 3 - 4
    return 0;
}

```

7. Python Implementation

```

def find_bridges(u, parent, adj, vis, tin, low, timer, bridges):
    vis[u] = True
    tin[u] = low[u] = timer[0]
    timer[0] += 1

    for v in adj[u]:
        if v == parent:
            continue

        if not vis[v]:
            find_bridges(v, u, adj, vis, tin, low, timer, bridges)
            low[u] = min(low[u], low[v])

            if low[v] > tin[u]:
                bridges.append((u, v))

```

```

        else:
            low[u] = min(low[u], tin[v])

def get_bridges(n, adj):
    vis = [False] * n
    tin = [-1] * n
    low = [-1] * n
    bridges = []
    timer = [0]

    for i in range(n):
        if not vis[i]:
            find_bridges(i, -1, adj, vis, tin, low, timer, bridges)

    return bridges

if __name__ == "__main__":
    n = 5
    adj = [[] for _ in range(n)]
    adj[0] = [1, 2]
    adj[1] = [0, 2, 3]
    adj[2] = [0, 1]
    adj[3] = [1, 4]
    adj[4] = [3]

    bridges = get_bridges(n, adj)
    print("Bridges:")
    for u, v in bridges:
        print(f"{u} - {v}")

```

8. Complexity Analysis

Operation	Time	Space
Find all bridges	$O(V + E)$	$O(V)$

9. Common Mistakes

- **Forgetting parent check** — causes false bridges (edge to parent looks like a bridge).
- **Using `tin[v]` instead of `low[v]` in bridge condition** — `low[v] > tin[u]` is correct.
- **Not handling multiple edges** — multiple edges between same nodes mean that edge is not a bridge.
- **Not handling disconnected graphs** — must run DFS from every unvisited node.

10. Practice Problems

Problem	Platform	Difficulty
1192. Critical Connections in a Network	LeetCode	Hard

Problem	Platform	Difficulty
Bridges in a graph	GFG	Medium

Articulation Points

1. Overview

An **articulation point** (or cut vertex) is a vertex whose removal increases the number of connected components in the graph. Like bridges, but for vertices.

2. Intuition

Think of a junction in a road network. If removing that junction disconnects the network, it's an articulation point.

Key insight: A vertex u is an articulation point if:

1. u is the root of the DFS tree and has **at least two children** in the DFS tree.
2. u is not the root and there exists a child v such that no vertex in v 's subtree has a back edge to an ancestor of u (i.e., $low[v] \geq tin[u]$).

3. When to Use It

- Finding critical nodes in a network
- Identifying single points of failure
- Network reliability analysis
- **Trigger phrases:** "articulation point", "cut vertex", "critical node"

4. Core Concepts

4.1 Root Case

- If root has > 1 child in DFS tree, it's an articulation point.
- **Why it matters:** Removing root disconnects its children.

4.2 Non-Root Case

- u is articulation point if there exists child v such that $low[v] \geq tin[u]$.
- **Why it matters:** Means the subtree of v has no connection to ancestors of u .

5. Step-by-Step Algorithm

1. Do DFS from any node.
2. Maintain $tin[u]$, $low[u]$, and track **children** count.
3. For each neighbour v of u :
 - If v is not visited:
 - Recurse.
 - Increment **children**.

- Update `low[u] = min(low[u], low[v])`.
- If `parent == -1 && children > 1`: `u` is articulation point.
- If `parent != -1 && low[v] >= tin[u]`: `u` is articulation point.
- Else if `v != parent`:
 - `low[u] = min(low[u], tin[v])`.

6. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void find_articulation_points(int u, int parent, vector<vector<int>> &adj,
                             vector<bool> &vis, vector<int> &tin, vector<int>
&low,
                             int &timer, vector<bool> &is_articulation) {
    vis[u] = true;
    tin[u] = low[u] = timer++;
    int children = 0;

    for (int v : adj[u]) {
        if (v == parent) continue;

        if (!vis[v]) {
            find_articulation_points(v, u, adj, vis, tin, low, timer,
is_articulation);
            low[u] = min(low[u], low[v]);
            children++;

            // Check articulation point conditions
            if (parent == -1 && children > 1) {
                is_articulation[u] = true;
            }
            if (parent != -1 && low[v] >= tin[u]) {
                is_articulation[u] = true;
            }
        } else {
            low[u] = min(low[u], tin[v]);
        }
    }
}

vector<int> get_articulation_points(int n, vector<vector<int>> &adj) {
    vector<bool> vis(n, false);
    vector<int> tin(n, -1), low(n, -1);
    vector<bool> is_articulation(n, false);
    int timer = 0;

    for (int i = 0; i < n; i++) {
        if (!vis[i]) {
            find_articulation_points(i, -1, adj, vis, tin, low, timer,
is_articulation);
        }
    }
}
```

```

    }

    vector<int> result;
    for (int i = 0; i < n; i++) {
        if (is_articulation[i]) result.push_back(i);
    }
    return result;
}

int main() {
    int n = 5;
    vector<vector<int>> adj(n);
    adj[0] = {1, 2};
    adj[1] = {0, 2, 3};
    adj[2] = {0, 1};
    adj[3] = {1, 4};
    adj[4] = {3};

    auto points = get_articulation_points(n, adj);
    cout << "Articulation points: ";
    for (int x : points) cout << x << " ";
    cout << "\n";
    // Output: 1 3
    return 0;
}

```

7. Python Implementation

```

def find_articulation_points(u, parent, adj, vis, tin, low, timer,
is_articulation):
    vis[u] = True
    tin[u] = low[u] = timer[0]
    timer[0] += 1
    children = 0

    for v in adj[u]:
        if v == parent:
            continue

        if not vis[v]:
            find_articulation_points(v, u, adj, vis, tin, low, timer,
is_articulation)
            low[u] = min(low[u], low[v])
            children += 1

        if parent == -1 and children > 1:
            is_articulation[u] = True
        if parent != -1 and low[v] >= tin[u]:
            is_articulation[u] = True
    else:
        low[u] = min(low[u], tin[v])

```

```
def get_articulation_points(n, adj):
    vis = [False] * n
    tin = [-1] * n
    low = [-1] * n
    is_articulation = [False] * n
    timer = [0]

    def find_articulation_points(i, parent, adj, vis, tin, low, timer, is_articulation):
        vis[i] = True
        tin[i] = low[i] = timer[0]
        timer[0] += 1
        children = 0
        for j in adj[i]:
            if j == parent:
                continue
            if not vis[j]:
                children += 1
                find_articulation_points(j, i, adj, vis, tin, low, timer, is_articulation)
                low[i] = min(low[i], low[j])
            else:
                low[i] = min(low[i], tin[j])
        if children > 1 or (parent != -1 and low[i] == tin[parent]):
            is_articulation[i] = True

    find_articulation_points(0, -1, adj, vis, tin, low, timer, is_articulation)

    return [i for i in range(n) if is_articulation[i]]

if __name__ == "__main__":
    n = 5
    adj = [[] for _ in range(n)]
    adj[0] = [1, 2]
    adj[1] = [0, 2, 3]
    adj[2] = [0, 1]
    adj[3] = [1, 4]
    adj[4] = [3]

    points = get_articulation_points(n, adj)
    print("Articulation points:", points)
```

8. Complexity Analysis

Operation	Time	Space
Find all articulation points	$O(V + E)$	$O(V)$

9. Common Mistakes

- **Root case special handling** — root is articulation point only if it has > 1 child in the DFS tree.
- **Using $> \text{tin}[u]$ (bridge condition) instead of $\geq \text{tin}[u]$** — articulation points use $\geq$.
- **Not handling the case where u is the root and has only 1 child** — that's not an articulation point.
- **Forgetting parent check** — leads to incorrect results.

10. Practice Problems

Problem	Platform	Difficulty
Articulation Points	GFG	Hard
Critical Nodes	SPOJ	Hard

Final Quick Reference

Complexity Comparison

Algorithm	Time	Space	Key Data Structure
BFS	$O(V+E)$	$O(V)$	Queue
DFS	$O(V+E)$	$O(V)$	Stack (recursion/explicit)
Connected Components	$O(V+E)$	$O(V)$	Visited array
Cycle Detection (Undirected)	$O(V+E)$	$O(V)$	Parent tracking
Cycle Detection (Directed)	$O(V+E)$	$O(V)$	3-state array
Bipartite Check	$O(V+E)$	$O(V)$	Colour array
Flood Fill	$O(\text{rows} \times \text{cols})$	$O(\text{rows} \times \text{cols})$	Stack/Queue
Matrix BFS	$O(\text{rows} \times \text{cols})$	$O(\text{rows} \times \text{cols})$	Queue
Multi-Source BFS	$O(\text{rows} \times \text{cols})$	$O(\text{rows} \times \text{cols})$	Queue (multiple sources)
0-1 BFS	$O(V+E)$	$O(V)$	Deque
DFS Timestamps	$O(V+E)$	$O(V)$	tin/tout arrays
Bridges	$O(V+E)$	$O(V)$	tin/low arrays
Articulation Points	$O(V+E)$	$O(V)$	tin/low arrays

Key Interview Tips

- **BFS:** Queue + visited[] + dist[] — shortest path in unweighted graphs
- **DFS:** Recursion + visited[] — explore all paths, cycle detection, topological sort
- **Bridges:** $\text{low}[v] > \text{tin}[u]$ — edge is critical
- **Articulation Points:** $\text{low}[v] \geq \text{tin}[u]$ (non-root) or $\text{children} > 1$ (root) — vertex is critical
- **Bipartite:** 2-colour with BFS/DFS — no odd cycle
- **0-1 BFS:** Deque — front for 0-weight, back for 1-weight
- **Multi-source BFS:** Push all sources first — nearest distance to any source
- **Cycle Directed:** 3-state array (0=unvisited, 1=in-stack, 2=done)
- **Cycle Undirected:** Parent check — `if (v != parent && vis[v])` → cycle